

Project 3: Dynamic Programming

Team 7

Members:

MADHAV RAGHU ANANTHARAM

GONG YICHEN

SEAN TEO WEI XIANG

LYE CHEE HUNG IRWIN

Unbounded Knapsack Problem Overview

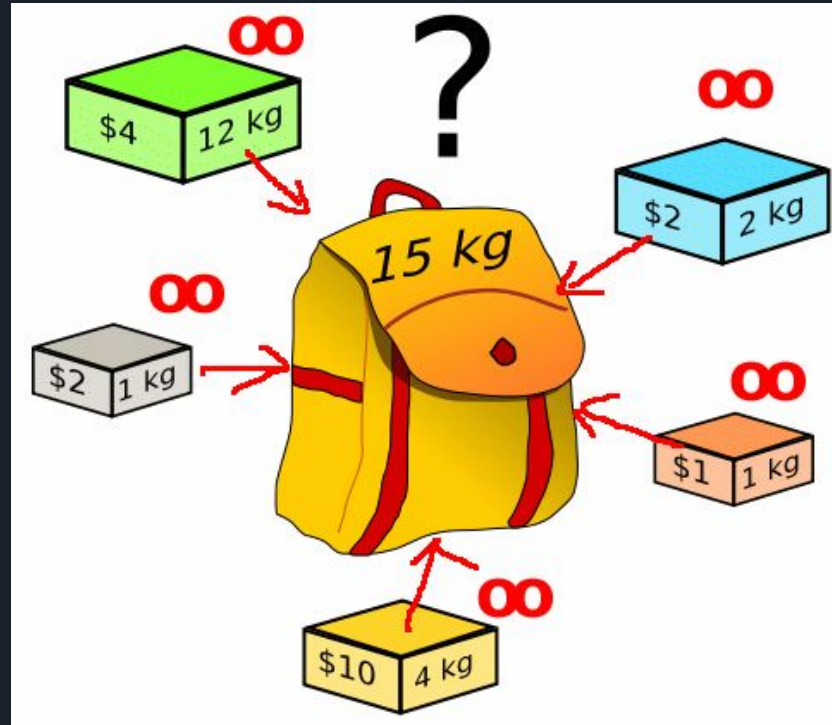


Fig1. Unbounded knapsack problem conceptual diagram



Unbounded Knapsack Problem Overview

Input Parameters	Constraints
• Knapsack capacity: C (a positive integer) • Number of item types: n • Weight array: $w = [w_0, w_1, \dots, w_{n-1}]$ where $w_i > 0$ • Profit array: $p = [p_0, p_1, \dots, p_{n-1}]$ where $p_i > 0$	• Unlimited supply of each item type • Total weight cannot exceed C: $\sum w_i x_i \leq C$
Objective	Mathematical Representation
• Maximize total profit	• $\max \{ \sum p_i x_i \mid \sum w_i x_i \leq C, x_i \in \mathbb{Z}_+ \}$



Problem Decomposition

1. Key question:

Which item should be the **last** one put into the knapsack?

2. Principle of Optimality:

If the optimal solution ends with item i , then:

The capacity $C - w_i$ must also achieve optimal profit $P(C - w_i)$

3. Optimal Decision:

$P(C) = \max \{$

$p_0 + P(C - w_0), \text{ (if } w_0 \leq C)$

$p_1 + P(C - w_1), \text{ (if } w_1 \leq C)$

...,

$p_{i-1} + P(C - w_{i-1}), \text{ (if } w_{i-1} \leq C)$

$0, \text{ (select no item)}$

$\}$



(1) Recursive definition of the function $P(C)$

Base Case:

$$P(0) = 0$$

// when the knapsack capacity is 0, no items can be included, so the profit is 0

Recursive Case:

$$P(C) = \max(0, \max\{P(C - w_i) + p_i \mid i \in [0, n-1] \text{ and } w_i \leq C\})$$

//If no items fit, i.e. all $w_i > c$, the set $\{P(C - w_i) + p_i \mid i \in [0, n-1] \text{ and } w_i \leq C\}$ is empty, so $P(C) = 0$



(2) Draw the subproblem graph for $P(14)$ where n is 3 with the weights and profits given below

	0	1	2
wi	4	6	8
pi	7	6	9

Recursive dependencies

$$\begin{aligned} P(C) = \max \{ & \\ & P(C-4) + 7, \text{ if } C \geq 4 \\ & P(C-6) + 6, \text{ if } C \geq 6 \\ & P(C-8) + 9, \text{ if } C \geq 8 \\ & 0 \quad \quad \quad (\text{base case}) \\ & \} \end{aligned}$$

Subproblem Computation for P(14)

Capacity	Calculation	Result
P(0)	Base case	0
P(1)–P(3)	No items fit	0
P(4)	$7+P(0)=7$	7
P(5)	$7+P(1)=7$	7
P(6)	$\max(7+P(2), 6+P(0))=\max(7, 6)$	7
P(7)	$\max(7+P(3), 6+P(1))=\max(7, 6)$	7
P(8)	$\max(7+P(4), 6+P(2), 9+P(0))=$ $\max(14, 6, 9)$	14

Subproblem Computation for P(14) - continued

Capacity	Calculation	Result
P(9)	$\max(7+P(5), 6+P(3), 9+P(1)) = \max(14, 6, 9)$	14
P(10)	$\max(7+P(6), 6+P(4), 9+P(2)) = \max(14, 13, 9)$	14
P(11)	$\max(7+P(7), 6+P(5), 9+P(3)) = \max(14, 13, 9)$	14
P(12)	$\max(7+P(8), 6+P(6), 9+P(4)) = \max(21, 13, 16)$	21
P(13)	$\max(7+P(9), 6+P(7), 9+P(5)) = \max(21, 13, 16)$	21
P(14)	$\max(7+P(10), 6+P(8), 9+P(6)) = \max(21, 20, 16)$	21

Subproblem Graph for P(14)

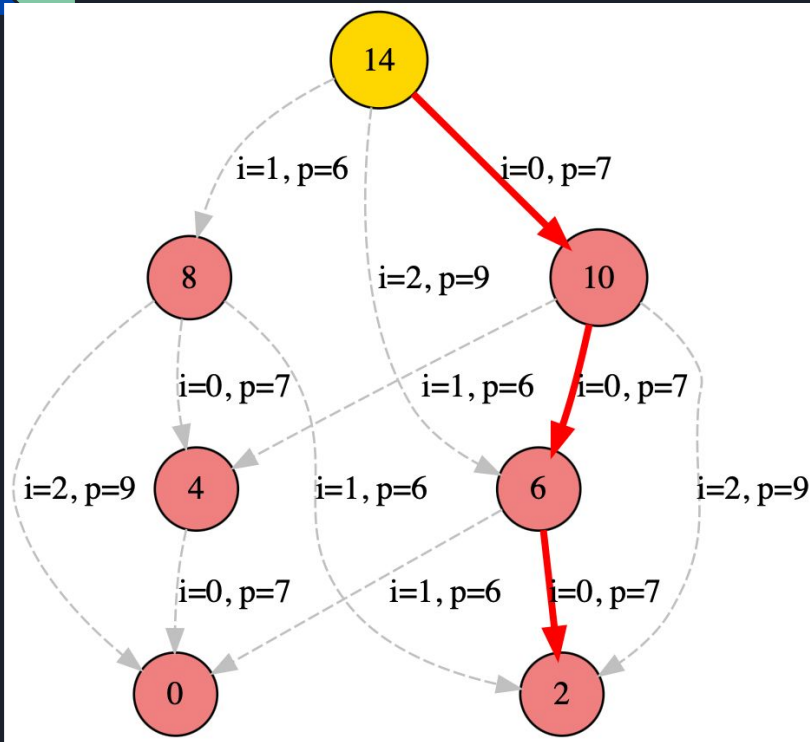


Fig2. P(14) subproblem graph

This graph shows **all subproblems visited** during the computation of P(14)

- Each node represents a subproblem P(C)
- **Red bold:** the optimal path
 - Profit: $7+7+7=21$
- **Dashed edges:** dependencies considered but not optimal

All weights are even, so only even capacities $\{0,2,4,6,8,10,14\}$ appear in the optimal dependency chain.

This illustrates the **principle of optimality**, since each subproblem on the optimal path is itself solved optimally.



(3) Dynamic programming algorithm to compute the maximum profit using the bottom up approach

Parameters:

- C: Knapsack capacity (positive integer)
- weights[]: Array of item weights $[w_0, w_1, \dots, w_{n-1}]$
- profits[]: Array of item profits $[p_0, p_1, \dots, p_{n-1}]$
- n: Number of item types

Output:

- Maximum profit achievable with capacity C

Time complexity:

- Worst case = Best case = Average case = $O(Cn)$

Space complexity:

- $O(C)$

Recursive Definition:

- Base Case: $P(0) = 0$
- Recursive Case: $P(C) = \max(0, \max\{P(C - w_i) + p_i \mid i \in [0, n-1] \text{ and } w_i \leq C\})$



The Algorithm

Create array `dp[0 .. C]` and initialize all elements to 0

// build DP table using bottom-up approach

for `c` in range (1, `C+1`):

 for `i` in range (length of weights):

 if `weights[i] ≤ c` :

`dp[c] = max(dp[c], dp[c - weights[i]] + profits[i])`

 end if

 end for

end for

print results

(4) Algorithm Implementation in Python



```
1  def max_profit(C, weights, profits):
2      dp = [0] * (C + 1)
3
4      for c in range(1, C+1):
5          for i in range(len(weights)):
6              if weights[i] <= c:
7                  dp[c] = max(dp[c], dp[c - weights[i]] + profits[i])
8
9      print(dp)
10     return dp[C]
11
```



a. P(14) with First Set of Weights



```
1 # First test case
2 weights1 = [4, 6, 8]
3 profits1 = [7, 6, 9]
4 C = 14
5 print("P(14) for first set:", max_profit(C, weights1, profits1))
6
```

Output:

```
[0, 0, 0, 0, 7, 7, 7, 7, 14, 14, 14, 14, 21, 21, 21]
P(14) for first set: 21
```



b. P(14) with Second Set of Weights



```
1 # Second test case
2 weights2 = [5, 6, 8]
3 profits2 = [7, 6, 9]
4 C = 14
5 print("P(14) for second set:", max_profit(C, weights2, profits2))
6
```

Output:

```
[0, 0, 0, 0, 0, 7, 7, 7, 9, 9, 14, 14, 14, 16, 16]
P(14) for second set: 16
```



Thank you!